

Sélection de variables

Analyse multidimensionnelle appliquée

Léo Belzile

HEC Montréal

1. Préparation de la base de données (analyse exploratoire, nettoyage)
2. Identification de la variable cible et des variables explicatives potentielles
3. Séparation de la base de données
4. Entraînement et estimation
 - a. Prétraitement des données
 - b. Choix du modèle
 - c. Choix des variables explicatives ou des hyperparamètres
5. Prédictions
6. Évaluation de la performance
7. Sélection du modèle final, ré-estimation et prédiction

On considère une base de données marketing. L'objectif d'affaires est de cibler les clients à qui envoyer un échantillon promotionnel.

But: maximiser les levées de fonds

1. Envoyer un échantillon de produits au coût de 10\$ à un groupe pilote de 1000 personnes et mesurer le montant d'achat suite à l'envoi.
2. Construire un modèle de prédiction pour déterminer à qui envoyer le produit parmi tous les 100 000 clients résiduels.

On supposera que, au final, chaque client a reçu l'échantillon. Le but de l'exercice rétrospectif est de voir s'il aurait été possible de faire mieux en ne ciblant que ceux susceptibles d'acheter.

Clients qui ont

- plus de 18 ans,
- au moins un an d'historique avec l'entreprise et
- qui ont effectué au moins un achat au cours de la dernière année.

Regroupements:

- 1K personnes dans l'échantillon d'apprentissage (pilote),
- 100K personnes pour l'ensemble des autres clients.

- `yachat`, une variable binaire qui indique si le client a acheté quelque chose suite à l'envoi, égale à 1 si oui et 0 sinon;
- `y montant`, le montant de l'achat si le client a acheté quelque chose;
- `x1`: sexe de l'individu, soit homme (0) ou femme (1);
- `x2`: l'âge (en année);
- `x3`: variable catégorielle indiquant le revenu, soit moins de 35 000\$ (1), entre 35 000\$ et 75 000\$ (2) ou plus de 75 000\$ (3);
- `x4`: variable catégorielle indiquant la région où habite le client (de 1 à 5);
- `x5`: couple : la personne est elle en couple (0=non, 1=oui);
- `x6`: nombre d'année du lien d'affaires;
- `x7`: nombre de semaines depuis le dernier achat;
- `x8`: montant (en dollars) du dernier achat;
- `x9`: montant total (en dollars) dépensé depuis un an;
- `x10`: nombre d'achats différents depuis un an.

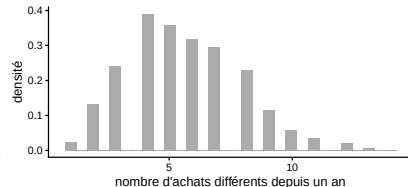
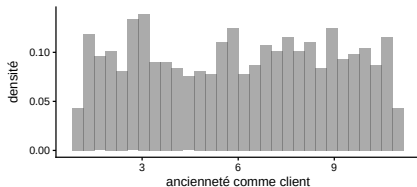
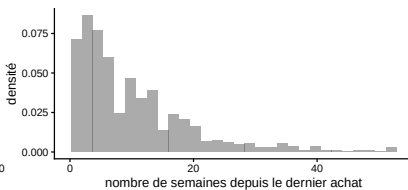
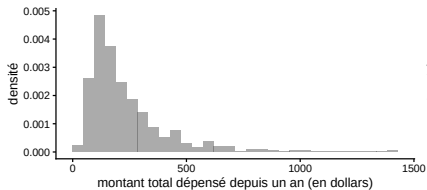
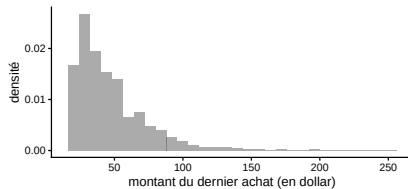
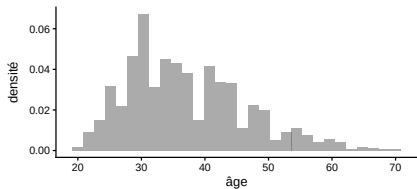
Aperçu des données

```
1 data(dbm, package = "hecmulti")
2 str(dbm)
```

```
tibble [101,000 x 13] (S3: tbl_df/tbl/data.frame)
 $ x1      : int [1:101000] 1 1 0 0 1 1 0 0 0 1 ...
 $ x2      : num [1:101000] 42 59 52 32 38 63 35 32 26 32 ...
 $ x3      : Factor w/ 3 levels "1","2","3": 1 2 3 1 2 2 2 1 3 1 ...
 $ x4      : Factor w/ 5 levels "1","2","3","4",...: 3 3 5 1 5 5 1 3 1 5 ...
 $ x5      : int [1:101000] 1 1 1 0 0 1 1 0 0 0 ...
 $ x6      : num [1:101000] 8.6 8.6 1.4 10.7 9.1 9.4 10.6 4.8 4 10.3 ...
 $ x7      : num [1:101000] 8 9 9 42 5 1 6 5 48 9 ...
 $ x8      : num [1:101000] 49 70 120 31 30 28 59 70 73 55 ...
 $ x9      : num [1:101000] 159 123 434 110 55 102 593 298 83 90 ...
 $ x10     : num [1:101000] 5 5 8 3 3 8 10 6 2 3 ...
 $ yachat  : int [1:101000] 0 0 0 0 0 0 0 1 1 1 ...
 $ ymontant: num [1:101000] NA NA NA NA NA NA NA 52 79 77 ...
 $ test    : Factor w/ 2 levels "0","1": 1 1 1 1 1 1 1 1 1 1 ...
```

sexe	décompte	couple	décompte
0	534	0	575
1	466	1	425

revenu	décompte	région	décompte
		1	216
1	397	2	185
2	337	3	216
3	266	4	191
		5	192



variable	moyenne	écart-type	min	max
x2	37.06	9.27	20	70
x6	6.01	2.92	1	11
x7	9.97	9.34	1	52
x8	48.41	28.27	20	252
x9	229.27	173.97	22	1407
x10	5.64	2.31	1	14

variable	description
x2	âge
x6	nombre d'année comme client
x7	nombre de semaines depuis le dernier achat
x8	montant du dernier achat
x9	montant total dépensé sur un an
x10	nombre d'achats différents sur un an

Le montant moyen dépensé s'écrit

$$E(y_{\text{montant}}) = E(y_{\text{montant}} \mid y_{\text{achat}} = 1) \Pr(y_{\text{achat}} = 1).$$

On bâtit un modèle de régression linéaire pour le montant moyen dépensé, $E(y_{\text{montant}} \mid y_{\text{achat}} = 1)$.

Le modèle utilise les données des 210 personnes (sur 1000) de l'échantillon d'apprentissage qui ont acheté suite à l'envoi de l'échantillon.

On conserve 100 000 observations test pour vérifier la performance

- (oracle): réponses inconnues à toutes fins pratiques en temps ordinaire.

Nous utiliserons la pénalisation ou la validation croisée pour choisir notre modèle, en nous basant uniquement sur les 1000 observations du projet pilote.

```
1 data(dbm, package = "hecmulti")
2 dbm_a <- dbm |>
3   dplyr::filter(
4     test == 0, #données d'entraînement
5     !is.na(ymontant)) # personnes qui ont acheté
```

Idée initiale:

- essayer tous les modèles possibles,
- estimer pour chacun la performance (par ex., l'erreur quadratique moyenne) par validation croisée,
- déterminer le meilleur modèle selon ce critère parmi les modèles candidats.

Les algorithmes que l'on va utiliser ont parfois des comportements différents face aux variables catégorielles (facteurs) avec $K > 2$ catégories, qui définit un ensemble de $K - 1$ variables binaires.

Soit le modèle considère l'ajout ou l'exclusion

- de chacune des $K - 1$ variables binaires créées,
- de l'ensemble des modalités simultanément.

La première option permet plus de flexibilité, mais comme les catégories retirées sont assimilées à l'ordonnée à l'origine, la catégorie de référence n'est plus arbitraire.

Combien de modèles incluant les combinaisons de p variables?
Chaque variable est incluse (ou pas): il y a $2^p = 2 \times 2 \times \dots \times 2$
(p fois) modèles.

Dans l'exemple, on a $p = 14$ variables de base en incluant les indicatrices pour les variables catégorielles multi-niveaux (revenu x3 et région x4), donc $2^{14} = 16384$ modèles.

p	nombre de modèles
5	32
10	1024
15	32768
20	1048576
25	33554432

Heureusement, il n'est pas nécessaire d'ajuster tous les modèles: certains peuvent être éliminés d'emblée.

L'algorithme par séparation et évaluation (branch and bound) recherche de manière efficace sans essayer tous les modèles candidats et écarte d'office les modèles sous-optimaux.

Implémenté avec critères d'information dans le paquet BranchGLM.

Essayer tous les modèles et choisir le meilleur (si p est petit).

```
1 # Recherche exhaustive avec variables de base
2 rec_ex <- leaps::regsubsets(
3   x = ymontant ~ x1 + x2 + x3 + x4 + x5 + x6 + x7 + x8 + x9 + x10,
4   nvmax = 13L,
5   method = "exhaustive",
6   data = dbm_a)
7 resume_rec_ex <- summary(rec_ex,
8                           matrix.logical = TRUE)
9 # Trouver le modèle avec le plus petit BIC
10 min_BIC <- which.min(resume_rec_ex$bic)
11 # Nom des variables dans le modèle retenu
12 rec_ex$xnames[resume_rec_ex$which[min_BIC,]]
```

```
[1] "(Intercept)" "x1"           "x32"          "x33"          "x45"
[6] "x5"           "x6"           "x7"           "x8"           "x10"
```

Quelques mantras pour la suite:

- préférer la flexibilité (réduire biais potentiel)
- être conscient de notre budget (surajustement)
- porter une attention particulière aux interactions entre variables catégorielles
 - estimations correspondent à des 'moyennes de groupe'
 - impact élevé potentiel des valeurs aberrantes et des extrêmes

On essaie d'estimer la moyenne de la variable cible y montant de la manière la plus flexible possible.

Pour ce faire, on pourrait aussi considérer tous les termes quadratiques, et les interactions d'ordre deux, pour obtenir plus de flexibilité:

- on n'inclut pas le carré de variables indicatrices binaires (car $0^2 = 0, 1^2 = 1$).
- idem pour les interactions entre indicateurs qui représentent des niveaux d'une même variable catégorielle.

Modèle complet et syntaxe R

```
1 # Dans R, x1:x2 crée une interaction entre deux variables
2 # (...)^2 crée toutes les interactions d'ordre deux
3 # I(x^2) permet de créer les termes quadratiques
4 formule <-
5     formula(ymontant ~
6             (x1 + x2 + x3 + x4 + x5 +
7              x6 + x7 + x8 + x9 + x10)^2 +
8              I(x2^2) + I(x6^2) + I(x7^2) +
9              I(x8^2) + I(x9^2) + I(x10^2))
10 mod_complet <- lm(formule, data = dbm_a)
11 # Matrice avec toutes les variables
12 matmod <- model.matrix(mod_complet)
```

Le modèle complet est clairement surajusté avec 105 coefficients pour 210 observations.

Recherche exhaustive typiquement trop coûteuse.

On peut plutôt opter pour un algorithme heuristique:

- à chaque étape, on maximise l'utilité (horizon d'optimisation limité) en retirant ou en ajoutant une seule variable.
- au début, p variables à regarder, puis il y a $p - 1$ choix l'étape suivante, etc.
- moins de modèles explorés, mais utile pour faire une recherche rapide.

Si on ajuste tous les modèles à K variables, celui qui a la plus petite erreur moyenne quadratique (EQM) a aussi le plus petit R^2 , AIC et BIC parmi ces modèles (puisque le nombre de coefficients est le même).

Il suffit donc de rapporter le meilleur modèle avec $1, 2, \dots, m$ variables explicatives et choisir ensuite.

On sélectionne un critère à minimiser (AIC/BIC) et une règle d'arrêt.

Sélection ascendante (forward selection): à partir du modèle de base (ordonnée à l'origine), ajouter à chaque étape au modèle précédent la variable qui réduit le plus notre critère d'optimisation.

Sélection descendante (backward selection): à partir du modèle de départ (typiquement le modèle complet), enlever la variable qui mène à la plus grande réduction du critère d'optimisation.

Dans les deux cas, la procédure se termine quand on ne peut plus réduire le critère.

On choisit le modèle final parmi ceux considérés avec le plus petit critère.

À partir du modèle de base (d'ordinaire),

- alterner sélection séquentielle ascendante et descendante.
- on continue ainsi tant que le modèle retourné par l'algorithme n'est pas identique à celui de l'étape précédente.
- une variable peut entrer dans le modèle et sortir plus tard dans le processus.
 - préférable aux procédures ascendantes et descendantes (car plus de modèles).
 - lui préférer la recherche exhaustive quand c'est possible

```
1 # Cette procédure séquentielle retourne
2 # la liste de modèles de 1 variable à
3 # nvmax variables.
4 rec_seq <-
5   leaps::regsubsets(
6     x = formule,
7     data = dbm_a,
8     method = "seqrep",
9     nvmax = length(coef(mod_complet))) # ici, 105
10 which.min(summary(rec_seq)$bic)
```

Sélection séquentielle avec critères d'information

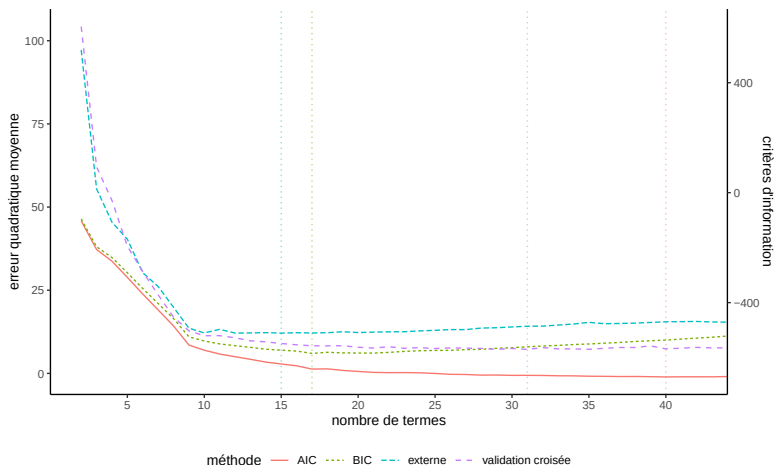
La procédure est plus longue à rouler (car les modèles linéaires sont ajustés).

On ajoute ou retire la variable qui améliore le plus le critère de sélection à chaque étape.

```
1 seq_AIC <- MASS::stepAIC(  
2   lm(ymontant ~ 1, data = dbm_a),  
3   # modèle initial sans variables explicative  
4   scope = formule, # modèle maximal possible  
5   direction = "both", #séquentielle  
6   trace = FALSE, # ne pas imprimer le suivi  
7   keep = function(mod, AIC, ...){  
8     # autres sorties des modèles à conserver  
9     list(bic = BIC(mod),  
10        coef = coef(mod))},  
11   k = 2) #  
12 # Remplacer k=2 par k = log(nrow(dbm_a)) pour BIC
```

L'historique des étapes est disponible via `seq_AIC$anova`

Performance en fonction de la complexité



Performance des 40 premiers modèles de la procédure séquentielle en fonction du nombre de termes inclus. La méthode externe donne la performance sur les 100K données de validation (oracle).

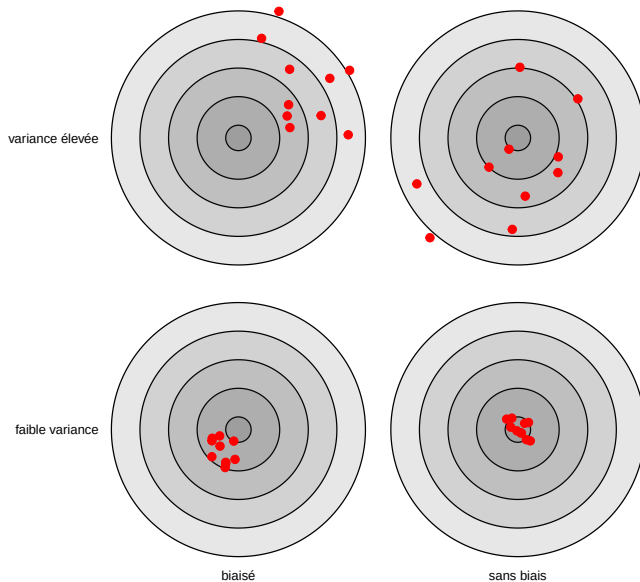
Objectif: prévenir le surajustement.

Si Y est l'observation, $\hat{Y}$ la prédiction et $E(Y)$ la vraie moyenne, l'erreur moyenne quadratique se décompose

$$\underset{\text{EQM}}{E \left[(Y - \hat{Y})^2 \right]} = E \left[\underbrace{\{\hat{Y} - E(Y)\}^2}_{\text{biais carré}} \right] + E \left[\underbrace{\{Y - E(Y)\}^2}_{\text{variabilité}} \right]$$

Les méthodes de régularisation introduisent du biais dans l'estimation des coefficients en pénalisant leur norme.

Compromis biais et variance



Compromis biais variance

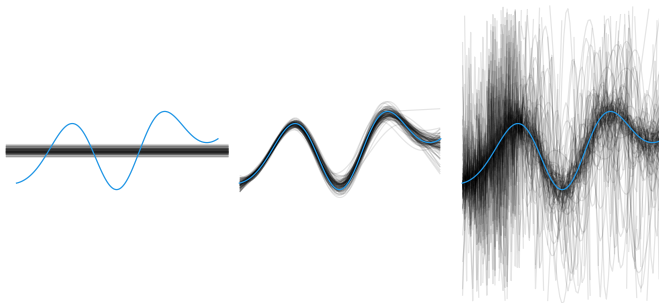


Figure 1: Illustration du compromis biais (gauche) et variance (droite). À mesure que l'on complexifie le modèle, l'ajustement s'améliore, mais le bruit finit par dominer.

Le modèle linéaire est invariant aux transformations affines.

Considérons un modèle dans lequel on changerait l'unité de la variable x_1

Si le coefficient estimé $\hat{\beta}_1 = 0.1$ quand x_1 est la distance en km, alors on aura $\hat{\beta}_1^* = 100 \times \hat{\beta}_1 = 10$ si on remplace cette variable par $x_1^* = x_1/100$ pour la distance exprimée en tranche de 100km.

La prédiction ne change pas car $\hat{\beta}_1 x_1 = \hat{\beta}_1^* x_1^*$.

On pénalise la norme de $\beta_1, \dots, \beta_p$ (l'ordonnée à l'origine n'est pas incluse).

Pour éviter que les échelles des variables (transformations affines) mènent à des changements, on standardise préalablement les variables explicatives $X_1, \dots, X_p$ et la variable réponse y pour qu'elles soient toutes à la même échelle (par exemple, avec moyenne nulle et un écart-type unitaire).

```
1 scale(dbm_a)
```

Cette étape peut-être effectuée implicitement par votre fonction ou logiciel (c'est le cas avec `glmnet` dans R).

Pénalité avec norme l_1 pour la valeur absolue des coefficients,

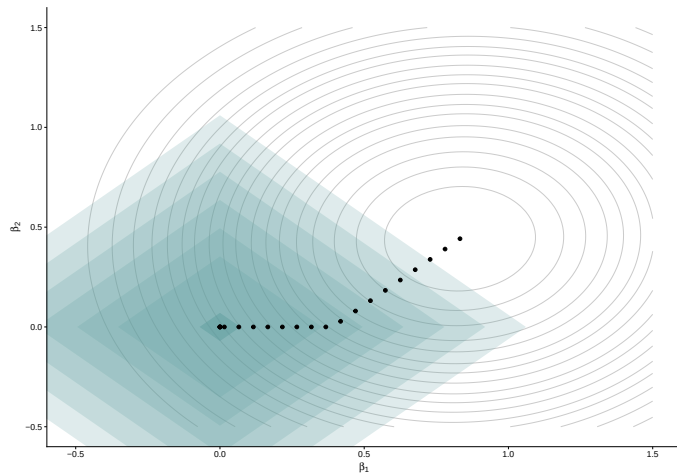
$$\min_{\beta} \{n\text{EQM}(\beta) + \lambda(|\beta_1| + \dots + |\beta_p|)\}.$$

L'hyperparamètre $\lambda > 0$ détermine la force de la pénalisation.

- Si $\lambda = \infty$, la solution est de prendre $\beta_1 = \dots = \beta_p = 0$ et on recouvre le modèle pour la moyenne globale, qui contient uniquement l'ordonnée à l'origine.
- Si $\lambda = 0$, on obtient le même résultat que le modèle linéaire avec les p variables explicatives.

Le LASSO rétrécit de certains coefficients exactement à zéro: sélection implicite de variable.

Contrainte budgétaire et moindres carrés

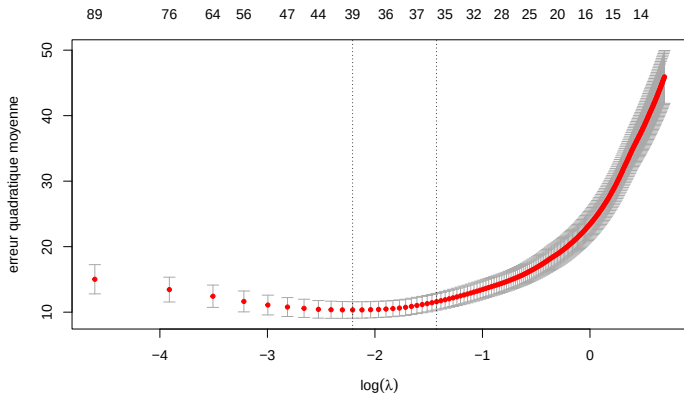


Paramètre de pénalité déterminé par validation croisée à partir d'un vecteur de valeurs candidates.

```
1 library(glmnet)
2 lambda_seq <- seq(from = 0.01, to = 2, by = 0.01)
3 cv_output <-
4   glmnet::cv.glmnet(x = as.matrix(matmod),
5                     y = dbm_a$ymontant,
6                     alpha = 1,
7                     lambda = lambda_seq)
8 plot(cv_output)
```

On choisit typiquement λ par validation croisée, soit

- le modèle avec la plus petite valeur de EQM_{VC} .
- le modèle le plus parsimonieux à au plus un erreur-type de ce dernier (pénalité plus élevée, plus grand nombre de coefficients nuls)



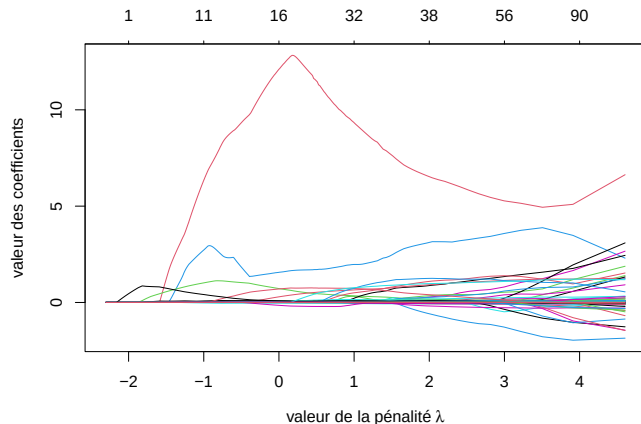


Figure 2: Valeur des coefficients du modèle en fonction de la pénalité (avec nombre de coefficients non-nuls sur le dessus).

Une fois la valeur de λ choisie, on ré-estime le modèle avec la pénalité.

```
1 lambdopt <- cv_output$lambda.min
2   #ou cv_output$lambda.1se
3 lasso_best <-
4   glmnet::glmnet(
5     x = as.matrix(as.matrix(matmod)),
6     y = dbm_a$ymontant,
7     alpha = 1,
8     lambda = lambdopt)
```

On crée une matrice avec les données de validation et on calcule l'erreur quadratique moyenne.

```
1 # Prédictions et calcul de l'EQM
2 # Données externes
3 dbm_v <- dbm |>
4   dplyr::filter(
5     test == 1,
6     !is.na(ymontant))
7 pred <- predict(lasso_best,
8                 s = lambopt,
9                 newx = as.matrix(
10                   model.matrix(formule,
11                                data = dbm_v)))
12 eqm_lasso <- mean((pred - dbm_v$ymontant)^2)
```

Il est important de distinguer entre la fonction objective (ou la fonction de coût) utilisée pour l'estimation des paramètres, comme

- l'erreur quadratique moyenne pour la régression linéaire
- le négatif de la log vraisemblance pour les modèles linéaires généralisés.

et le critère visant à choisir le meilleur modèle (par ex., le profit).

Évaluation de la performance

En incluant uniquement les variables de base

nb variables	EQM	méthode
15	25.69	toutes les variables
12	25.53	exhaustive - AIC
10	25.04	exhaustive - BIC

En incluant les termes de base, les carrés et les interactions d'ordre 2.

nb variables	EQM	méthode
104	19.63	toutes les variables
21	12	séquentielle, choix selon AIC
15	12.31	séquentielle, choix selon BIC
15	13.64	ridge, validation croisée avec 10 groupes
30	11.94	LASSO, validation croisée avec 10 groupes

- Le nombre de modèles possibles augmente rapidement avec le nombre de prédicteurs.
- Si un modèle est mal spécifié (variables importantes manquantes), alors les estimations sont biaisées.
- Si le modèle est surspécifié, les coefficients correspondants aux variables superflues incluses sont en moyenne nuls, mais contribuent à l'augmentation de la variance.
- Compromis biais/variance.

- La taille du modèle (le nombre de variables explicatives) est restreinte par le nombre d'observations disponibles.
- En général, il faut s'assurer d'avoir suffisamment d'observations pour estimer de manière fiable les coefficients.
- Porter une attention particulière aux variables binaires et aux interactions avec ces dernières: si les effectifs de certaines modalités sont faibles, il y a danger de surajustement.

- Une recherche exhaustive garantie le survol du plus grand nombre de modèles possibles, mais est coûteuse
- On peut effectuer une recherche exhaustive à l'aide d'algorithmes d'optimisation (séparation et évaluation) pour un nombre réduit de variables (max 50).
- Sinon, on a l'option d'utiliser un algorithme heuristique qui ne couvre qu'un sous-ensemble de tous les modèles
- Compromis coût de calcul vs nombre de modèles explorés
- Possibilité de combiner des méthodes!

Certaines méthodes de pénalisation directe changent la fonction objective:

- introduction de biais pour les coefficients (vers zéro) pour réduire la variabilité des estimations.
- contrairement aux critères d'information (qui pénalisent l'ajustement a posteriori), les estimations des coefficients changent.

Une pénalité particulière (LASSO) contraint certains paramètres à être exactement nuls.

- Cela correspond implicitement à une sélection de variables, bien que les estimations sont néanmoins différentes de celle des moindres carrés (régression linéaire) pour les mêmes variables explicatives.

- En pratique, on cherche à essayer le plus possible de bons modèles pour trouver le choix optimal de variables.
- Faire un tri permet d'isoler certains candidats, pour lesquels on peut calculer notre critère de sélection.
- On retient alors celui qui donne la meilleure performance.